**UNIVERSIDAD POLITÉCNICA DE ATLAUTLA**
Ing. en Tecnologías de la Información e Innovación Digital

**ACTIVIDAD PARCIAL 2 · SECCIÓN 6 · PROYECTO FINAL**

# Algoritmo genético

Programación de turnos de personal con una población evolutiva (DEAP), 30 generaciones.

2.6 Algoritmos genéticos · Proyecto en equipo

**INTEGRANTES DEL EQUIPO**

- **Bernal Najera Alexandra**
- **Marin Flor Julissa**
- **Rosas Toriz Erandy Naomy**
- **Suarez Melonie Magdalena**

**DOCENTE**
Mtro. Javier García Lara

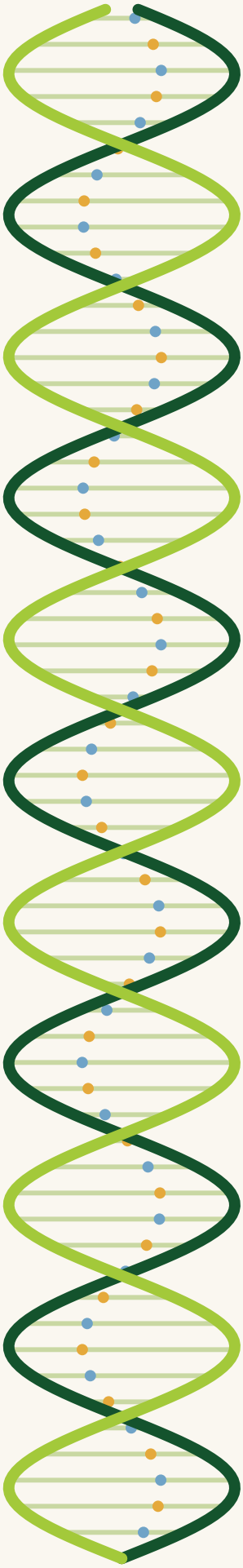
**GRUPO**
171 · Cuatrimestre 7

**PERIODO**
12 al 26 de octubre de 2026

**ENTREGA**
26 de octubre de 2026 · Equipo

**ASIGNATURA**
Fundamentos de Inteligencia Artificial

**REPOSITORIO**



SECCIÓN 1

# Planteamiento del problema

Un negocio con servicio extendido (por ejemplo, una clínica o una tienda) debe asignar a 8 empleados un turno por día durante una semana: mañana, tarde, noche o descanso. El horario debe cubrir el personal mínimo de cada turno y respetar reglas laborales. Con 8 empleados, 7 días y 4 opciones por día hay $4^{56} \approx 5.2 \times 10^{33}$ horarios posibles, así que probarlos todos es imposible.

Un **algoritmo genético (AG)** resuelve este tipo de problemas imitando la evolución: mantiene una población de soluciones, elige las mejores, las combina y las muta durante varias generaciones (Holland, 1975; Goldberg, 1989). Se implementa con la librería DEAP (Fortin et al., 2012).

**PREGUNTA GUÍA**
¿Puede un AG de 30 generaciones encontrar horarios que cumplan la cobertura y reduzcan las violaciones, y cuánto mejora frente a una búsqueda aleatoria?

## Objetivos

- Diseñar el cromosoma, la función de aptitud y los operadores genéticos para un problema real.
- Ejecutar el AG durante 30 generaciones y graficar su convergencia.
- Comparar con una búsqueda aleatoria, analizar la sensibilidad a la tasa de mutación y documentar cómo replicar los resultados.

SECCIÓN 2

# Diseño del algoritmo genético

| Componente | Diseño |
|---|---|
| **Cromosoma** | Lista de 56 genes (8 empleados × 7 días). El gen en la posición e·7 + d es el turno del empleado e el día d. |
| **Gen** | Entero: 0 = descanso, 1 = mañana, 2 = tarde, 3 = noche. |
| **Población** | 100 individuos generados al azar (el descanso, la mañana y la tarde son tres veces más probables que la noche). |
| **Fitness (aptitud)** | Penalización total por violaciones de las reglas. Se **minimiza**; 0 sería un horario perfecto. |
| **Selección** | Torneo de tamaño 3: de 3 individuos al azar pasa el de menor penalización. |
| **Cruce** | Por empleado, con probabilidad 0.8: dos padres intercambian filas completas (el horario semanal de un empleado), lo que conserva la estructura de cada fila. |
| **Mutación** | Con probabilidad 0.3 por individuo; cada gen cambia a un turno al azar con probabilidad 0.04. |
| **Elitismo** | Los 2 mejores individuos pasan sin cambios a la siguiente generación. |
| **Generaciones** | 30. |

Tabla 1. Componentes del algoritmo genético.

## Función de aptitud: reglas y pesos

| Regla | Se penaliza cuando… | Peso | Unidad |
|---|---|---|---|
| **cobertura** | Faltan personas en un turno (mínimo: 2 mañana, 2 tarde, 1 noche por día) | 10 | por persona faltante |
| **sobrecobertura** | Sobran más de 1 persona sobre el mínimo | 2 | por persona de más |
| **dias max** | Trabaja más de 5 días en la semana | 8 | por día de más |
| **dias min** | Trabaja menos de 4 días | 3 | por día de menos |
| **noche manana** | Turno de noche seguido de turno de mañana (sin descanso) | 6 | por caso |
| **sin noche** | Carla o Fabián asignados a turno de noche (no pueden) | 15 | por turno |
| **noches max** | Más de 3 noches en la semana | 5 | por noche de más |
| **equidad noches** | Noches mal repartidas entre empleados | 1 | por la varianza |

Tabla 2. Fitness = suma de (peso × violación). Los pesos altos corresponden a reglas que no deben romperse (cobertura y restricciones de noche).

SECCIÓN 3

# Datos y procedimiento

## Datos del caso

| Dato | Valor |
|---|---|
| **Empleados** | Ana, Beto, Carla, Diego, Elena, Fabián, Gaby, Hugo |
| **Días y turnos** | Lunes a domingo; turnos Mañana, Tarde, Noche y Descanso. |
| **Restricciones personales** | Carla y Fabián no pueden trabajar de noche. |
| **Capacidad** | Cobertura mínima de 5 personas por día; 8 empleados × 5 días máximos = 40 turnos disponibles frente a 35 requeridos. |

Tabla 3. Los datos son un caso de ejemplo definido para la actividad, no de una empresa real.

## Ciclo evolutivo

```
poblacion <- 100 horarios aleatorios ; evaluar fitness
repetir 30 generaciones:
    elite     <- los 2 mejores
    hijos     <- torneo(poblacion, 98 individuos)
    cruce por empleado entre pares de hijos (prob. 0.8)
    mutacion de cada hijo (prob. 0.3; 4 % de sus genes)
    evaluar fitness de los hijos modificados
    poblacion <- hijos + elite
devolver el mejor individuo
```

*Pseudocódigo del ciclo evolutivo (el código completo está en el Anexo A).*

## Instructivo para replicar los resultados

| Paso | Acción |
|---|---|
| **1** | Instalar Python 3.10 o superior. |
| **2** | Instalar las librerías: `pip install deap numpy matplotlib` |
| **3** | Descargar la carpeta `S6_algoritmo_genetico` del repositorio. |
| **4** | Ejecutar: `python s6_algoritmo_genetico.py` (tarda menos de un minuto). |
| **5** | Revisar las gráficas en `figuras/` y los datos en `resultados_s6.json`. |
| **6** | Para probar otros casos, cambiar en el código los empleados, la cobertura o los pesos. |

Las semillas aleatorias están fijas (0 a 9 y 42), por lo que se obtienen los mismos resultados.

SECCIÓN 4

# Resultados

## 4.1 Convergencia

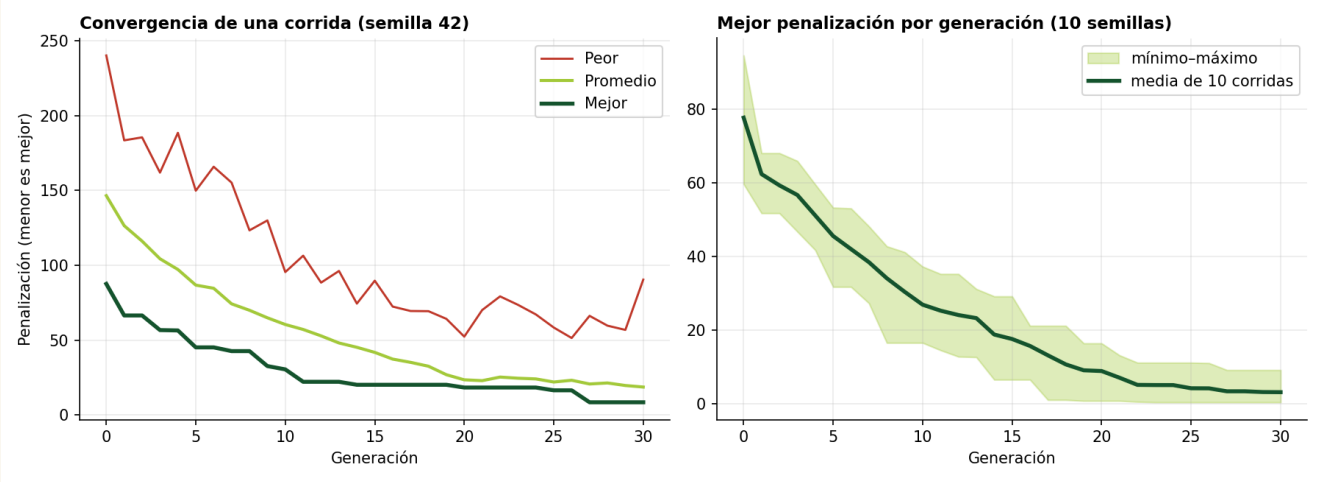



*Figura 1. Izquierda: mejor, promedio y peor penalización por generación (semilla 42). Derecha: mejor penalización en 10 corridas con distintas semillas.*

| Resultado | Valor |
|---|---|
| **Corrida semilla 42** | mejor penalización 87.6 (generación 0) → **8.5** (generación 30) |
| **10 corridas del AG** | media **3.1** · desviación 3.7 · mejor 0.4 · peor 9.1 |
| **10 búsquedas aleatorias (3,040 evaluaciones cada una)** | media 50.7 · mejor 41.4 |

Tabla 4. Penalización final. La búsqueda aleatoria usa el mismo número de evaluaciones que el AG (3,040); es decir, ambos tuvieron el mismo esfuerzo de cálculo.

## 4.2 Mejor horario encontrado

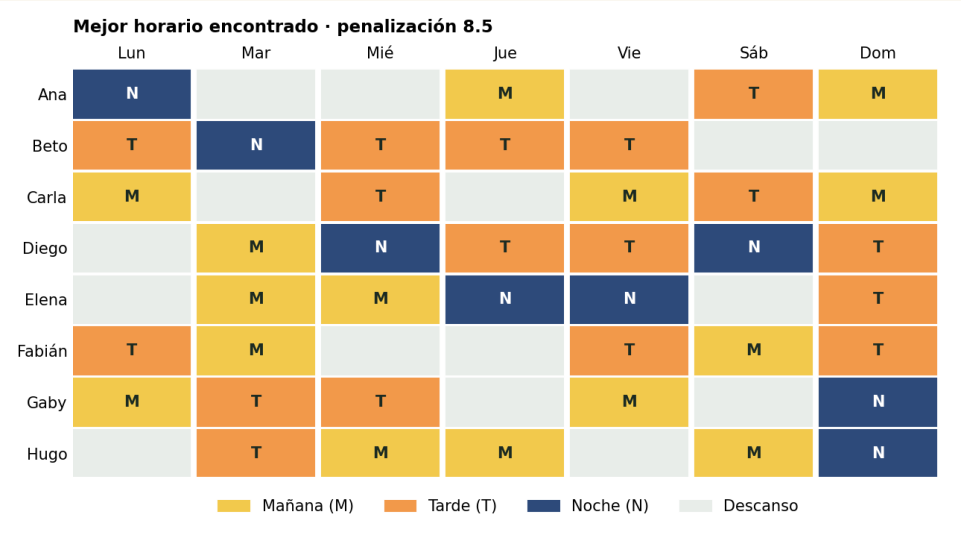



*Figura 2. Horario de la corrida con semilla 42 (M = mañana, T = tarde, N = noche; casilla gris = descanso).*

| Regla | Violación (sin ponderar) | Peso | Aporte a la penalización |
|---|---|---|---|
| **cobertura** | 0.00 | 10 | 0.0 |
| **sobrecobertura** | 0.00 | 2 | 0.0 |
| **dias max** | 1.00 | 8 | 8.0 |
| **dias min** | 0.00 | 3 | 0.0 |
| **noche manana** | 0.00 | 6 | 0.0 |
| **sin noche** | 0.00 | 15 | 0.0 |
| **noches max** | 0.00 | 5 | 0.0 |
| **equidad noches** | 0.50 | 1 | 0.5 |
| **Total** | | | **8.5** |

Tabla 5. Desglose de la penalización del mejor horario: cumple toda la cobertura y las restricciones de noche; solo queda una violación de días máximos y un reparto de noches casi parejo.

### 4.3 Sensibilidad a la tasa de mutación

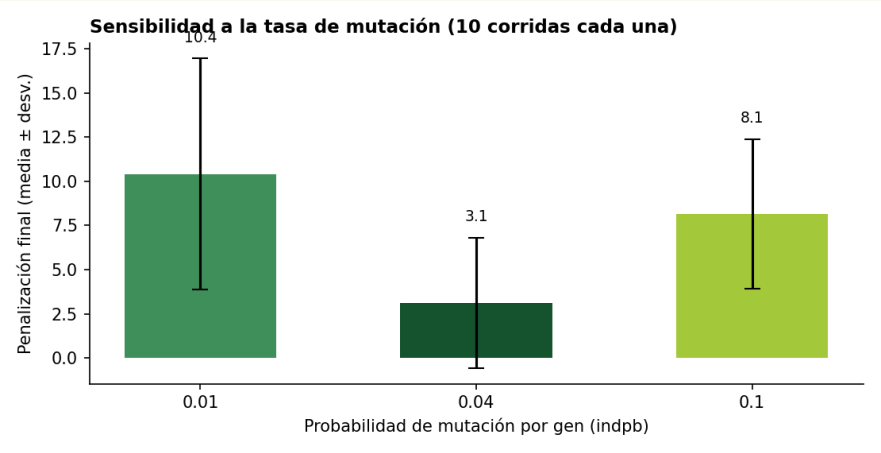



*Figura 3. Penalización final según la probabilidad de mutación por gen (10 corridas cada una).*

SECCIÓN 5

## Análisis de resultados

- **El AG mejora mucho frente al azar.** Con el mismo número de evaluaciones, la penalización promedio bajó de 50.7 (búsqueda aleatoria) a 3.1 (AG), unas 16 veces menos. La evolución aprovecha lo bueno de los padres en lugar de probar a ciegas.
- **La mayor parte de la mejora ocurre en las primeras generaciones.** En la corrida de la semilla 42 la penalización pasó de 87.6 a 8.5; la curva baja rápido al inicio y luego se aplana (Figura 1).
- **Hay variabilidad entre corridas.** Las 10 corridas terminaron entre 0.4 y 9.1, por la naturaleza aleatoria del algoritmo. Por eso se reportan promedios y no una sola corrida.
- **La tasa de mutación importa.** Con 0.01 el AG explora poco (media 10.4); con 0.1 destruye demasiado las buenas soluciones (media 8.1); 0.04 dio el mejor equilibrio (media 3.1), que es el valor usado.
- **Qué no se puede afirmar.** El resultado es el mejor encontrado en 30 generaciones, no se demostró que sea el óptimo. Con más generaciones probablemente mejoraría, pero eso no se probó. Los pesos de las reglas son una decisión del equipo y cambiarlos cambiaría el horario resultante.
- **Uso real.** Un horario así sirve como propuesta que la persona encargada revisa y ajusta, no como decisión automática: el algoritmo no conoce preferencias ni imprevistos que no estén en la función de aptitud.

SECCIÓN 6

## Conclusiones reflexivas

- El diseño de la función de aptitud es la parte más importante del AG: traduce las reglas del negocio a un número que el algoritmo puede minimizar.
- Un cruce que respeta la estructura del problema (intercambiar el horario completo de un empleado) fue más natural que mezclar genes sueltos.
- Los AG son útiles cuando el espacio es enorme y no existe una fórmula directa, como en la programación de horarios; a cambio no garantizan el óptimo y requieren ajustar parámetros.

- Este problema conecta con los anteriores: es una búsqueda local (S3) trabajada con una población de soluciones en lugar de una sola.
- Limitaciones: caso de ejemplo con 8 empleados, una sola semana y restricciones simplificadas; un caso real incluiría vacaciones, preferencias, contratos y días festivos.

SECCIÓN 7

## Declaración de uso de inteligencia artificial

| | |
|---|---|
| **Herramienta** | Claude (Anthropic, 2026), modelo Claude Sonnet 5.5, con ejecución de código Python. |
| **Prompt (resumen)** | Se entregó la hoja de la Actividad Parcial 2 y se pidió el código y el PDF de la sección 6 (2.6 Algoritmos genéticos con DEAP, proyecto final en equipo). |
| **Uso** | Diseño del caso, el cromosoma y la función de aptitud; escritura del código (AG con DEAP, experimentos y gráficas); borrador del texto y maquetación del PDF (reportlab). |
| **Verificación** | El código se ejecutó completo y el desglose de la penalización se comprobó contra la penalización total del mejor individuo. El caso y los datos son de ejemplo. Las referencias deben cotejarse con las fuentes originales antes de entregar. |

SECCIÓN R

## Referencias (APA 7.ª ed.)

Anthropic. (2026). *Claude Sonnet 5.5* [Modelo de lenguaje de gran tamaño]. https://claude.ai

Fortin, F.-A., De Rainville, F.-M., Gardner, M.-A., Parizeau, M., & Gagné, C. (2012). DEAP: Evolutionary algorithms made easy. *Journal of Machine Learning Research, 13*, 2171-2175.

Goldberg, D. E. (1989). *Genetic algorithms in search, optimization, and machine learning*. Addison-Wesley.

Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature, 585*, 357-362. https://doi.org/10.1038/s41586-020-2649-2

Holland, J. H. (1975). *Adaptation in natural and artificial systems*. University of Michigan Press.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90-95. https://doi.org/10.1109/MCSE.2007.55

Russell, S., & Norvig, P. (2020). *Inteligencia Artificial: Un enfoque moderno* (4.ª ed.). Pearson.

Wirsansky, E. (2020). *Hands-on genetic algorithms with Python*. Packt.

SECCIÓN A

# Anexo A. Código Python

Archivo: s6_algoritmo_genetico.py

```
"""S6 · Algoritmo genético (DEAP) para programar los turnos semanales de un equipo de trabajo

Caso empresarial: un negocio con servicio extendido (por ejemplo, una clínica o una tienda) debe
asignar a 8 empleados un turno por día durante una semana. El algoritmo genético busca el horario
que cumple la cobertura mínima y reduce las violaciones de las reglas laborales.

  Cromosoma : lista de 56 genes (8 empleados x 7 días); cada gen es 0=descanso, 1=mañana, 2=tarde, 3=noche
  Fitness   : penalización total a MINIMIZAR (0 = horario perfecto)
  Operadores: selección por torneo, cruce por empleado, mutación uniforme y elitismo
  Corrida   : población de 100 individuos y 30 generaciones

Equipo: Bernal Najera Alexandra · Marin Flor Julissa · Rosas Toriz Erandy Naomy · Suarez Melonie Magdalena
Ejecutar: python s6_algoritmo_genetico.py
Requiere: pip install deap numpy matplotlib
Genera:   figuras/convergencia.png, figuras/horario_optimo.png, figuras/sensibilidad.png y resultados_s6.json
"""
import json
import os
import random

import matplotlib
matplotlib.use("Agg")  # genera PNG sin abrir ventana
import matplotlib.pyplot as plt
import numpy as np
from deap import base, creator, tools

AQUI = os.path.dirname(os.path.abspath(__file__))
FIG = os.path.join(AQUI, "figuras")
os.makedirs(FIG, exist_ok=True)

# ============================================================================
# 1. Datos del problema
# ============================================================================
EMPLEADOS = ["Ana", "Beto", "Carla", "Diego", "Elena", "Fabián", "Gaby", "Hugo"]
DIAS = ["Lun", "Mar", "Mié", "Jue", "Vie", "Sáb", "Dom"]
TURNOS = {0: "Descanso", 1: "Mañana", 2: "Tarde", 3: "Noche"}
NE, ND = len(EMPLEADOS), len(DIAS)
LARGO = NE * ND                                 # genes del cromosoma
COBERTURA = {1: 2, 2: 2, 3: 1}                  # personas mínimas por turno y día
MAX_DIAS, MIN_DIAS, MAX_NOCHES = 5, 4, 3        # días trabajados por semana y noches por semana
SIN_NOCHE = {2, 5}                              # Carla y Fabián no pueden trabajar de noche
PESOS = {"cobertura": 10, "sobrecobertura": 2, "dias_max": 8, "dias_min": 3,
         "noche_manana": 6, "sin_noche": 15, "noches_max": 5, "equidad_noches": 1}

# Parámetros del algoritmo genético
POBLACION, GENERACIONES = 100, 30
CXPB, MUTPB, INDPB, TORNEO, ELITE = 0.8, 0.3, 0.04, 3, 2


def matriz(ind):
    """Convierte el cromosoma plano en una matriz empleados x días."""
    return np.array(ind, dtype=int).reshape(NE, ND)


# ============================================================================
# 2. Función de aptitud (fitness): suma ponderada de violaciones
# ============================================================================
def penalizaciones(ind):
    m = matriz(ind)
    p = dict.fromkeys(PESOS, 0.0)
    for d in range(ND):
        for t, minimo in COBERTURA.items():
            n = int(np.sum(m[:, d] == t))
            p["cobertura"] += max(0, minimo - n)              # faltan personas
            p["sobrecobertura"] += max(0, n - (minimo + 1))    # sobran personas
    trabajados = (m > 0).sum(axis=1)
    p["dias_max"] = float(np.sum(np.maximum(0, trabajados - MAX_DIAS)))
    p["dias_min"] = float(np.sum(np.maximum(0, MIN_DIAS - trabajados)))
    p["noche_manana"] = float(np.sum((m[:, :-1] == 3) & (m[:, 1:] == 1)))   # sin descanso suficiente
    p["sin_noche"] = float(sum(np.sum(m[e] == 3) for e in SIN_NOCHE))
    noches = (m == 3).sum(axis=1)
```

```
    p["noches_max"] = float(np.sum(np.maximum(0, noches - MAX_NOCHES)))
    p["equidad_noches"] = float(np.var(noches))
    return p


def aptitud(ind):
    p = penalizaciones(ind)
    return (sum(PESOS[k] * v for k, v in p.items()),)


# ======================================================================
# 3. Operadores genéticos
# ======================================================================
def cruce_por_empleado(a, b):
    """Intercambia filas completas (el horario de un empleado) entre dos padres."""
    for e in range(NE):
        if random.random() < 0.5:
            i = e * ND
            a[i:i + ND], b[i:i + ND] = b[i:i + ND], a[i:i + ND]
    return a, b


if not hasattr(creator, "FitnessMin"):
    creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
    creator.create("Individuo", list, fitness=creator.FitnessMin)


def crear_toolbox(indpb=INDPB):
    tb = base.Toolbox()
    tb.register("gen", lambda: random.choices([0, 1, 2, 3], weights=[3, 3, 3, 1])[0])
    tb.register("individuo", tools.initRepeat, creator.Individuo, tb.gen, LARGO)
    tb.register("poblacion", tools.initRepeat, list, tb.individuo)
    tb.register("evaluate", aptitud)
    tb.register("select", tools.selTournament, tournsize=TORNEO)
    tb.register("mate", cruce_por_empleado)
    tb.register("mutate", tools.mutUniformInt, low=0, up=3, indpb=indpb)
    return tb


# ======================================================================
# 4. Ciclo evolutivo (con elitismo)
# ======================================================================
def evolucionar(semilla=42, n=POBLACION, ngen=GENERACIONES, indpb=INDPB):
    random.seed(semilla)
    tb = crear_toolbox(indpb)
    pop = tb.poblacion(n=n)
    for ind in pop:
        ind.fitness.values = tb.evaluate(ind)
    hist = []

    def registrar(g):
        f = [i.fitness.values[0] for i in pop]
        hist.append({"gen": g, "mejor": min(f), "promedio": float(np.mean(f)), "peor": max(f)})

    registrar(0)
    for g in range(1, ngen + 1):
        elite = [tb.clone(i) for i in tools.selBest(pop, ELITE)]
        hijos = [tb.clone(i) for i in tb.select(pop, n - ELITE)]
        for a, b in zip(hijos[::2], hijos[1::2]):
            if random.random() < CXPB:
                tb.mate(a, b)
                del a.fitness.values, b.fitness.values
        for h in hijos:
            if random.random() < MUTPB:
                tb.mutate(h)
                del h.fitness.values
        for h in hijos:
            if not h.fitness.valid:
                h.fitness.values = tb.evaluate(h)
        pop[:] = hijos + elite
        registrar(g)
    mejor = tools.selBest(pop, 1)[0]
    return mejor, hist


def busqueda_aleatoria(semilla, evaluaciones):
    """Línea base: mismo número de evaluaciones que el AG, pero sin evolución."""
    random.seed(semilla)
    tb = crear_toolbox()
```

```
    return min(tb.evaluate(tb.individuo())[0] for _ in range(evaluaciones))


# ======================================================================
# 5. Experimentos y gráficas
# ======================================================================
VERDE, VERDE2, LIMA, CREMA, TINTA = "#14532D", "#3F8F5A", "#A3C93A", "#FAF7F0", "#1B2A22"
COLORES_TURNO = {0: "#E8EDE9", 1: "#F2C94C", 2: "#F2994A", 3: "#2D4A7A"}


def grafica_convergencia(hist, corridas):
    fig, ax = plt.subplots(1, 2, figsize=(12, 4.4))
    g = [h["gen"] for h in hist]
    ax[0].plot(g, [h["peor"] for h in hist], color="#C0392B", lw=1.4, label="Peor")
    ax[0].plot(g, [h["promedio"] for h in hist], color=LIMA, lw=2, label="Promedio")
    ax[0].plot(g, [h["mejor"] for h in hist], color=VERDE, lw=2.6, label="Mejor")
    ax[0].set_title("Convergencia de una corrida (semilla 42)", fontsize=11, fontweight="bold", loc="left")
    ax[0].set_xlabel("Generación"); ax[0].set_ylabel("Penalización (menor es mejor)"); ax[0].legend()
    M = np.array([[h["mejor"] for h in c] for c in corridas])
    x = np.arange(M.shape[1])
    ax[1].fill_between(x, M.min(0), M.max(0), color=LIMA, alpha=0.35, label="mínimo–máximo")
    ax[1].plot(x, M.mean(0), color=VERDE, lw=2.6, label=f"media de {len(corridas)} corridas")
    ax[1].set_title("Mejor penalización por generación (10 semillas)", fontsize=11, fontweight="bold", loc="left")
    ax[1].set_xlabel("Generación"); ax[1].legend()
    for a in ax:
        a.spines[["top", "right"]].set_visible(False); a.grid(alpha=0.25)
    fig.tight_layout(); fig.savefig(os.path.join(FIG, "convergencia.png"), dpi=150); plt.close(fig)


def grafica_horario(mejor, etiqueta):
    m = matriz(mejor)
    fig, ax = plt.subplots(figsize=(8.6, 4.8))
    for e in range(NE):
        for d in range(ND):
            ax.add_patch(plt.Rectangle((d, NE - 1 - e), 0.94, 0.92, color=COLORES_TURNO[m[e, d]]))
            txt = {0: "", 1: "M", 2: "T", 3: "N"}[m[e, d]]
            ax.text(d + 0.47, NE - 1 - e + 0.46, txt, ha="center", va="center", fontsize=10, fontweight="bold",
                    color="white" if m[e, d] == 3 else TINTA)
    ax.set_xlim(0, ND); ax.set_ylim(0, NE)
    ax.set_xticks([d + 0.47 for d in range(ND)]); ax.set_xticklabels(DIAS)
    ax.set_yticks([NE - 1 - e + 0.46 for e in range(NE)]); ax.set_yticklabels(EMPLEADOS)
    ax.xaxis.tick_top(); ax.tick_params(length=0)
    for s in ax.spines.values():
        s.set_visible(False)
    handles = [plt.Rectangle((0, 0), 1, 1, color=COLORES_TURNO[k]) for k in (1, 2, 3, 0)]
    ax.legend(handles, ["Mañana (M)", "Tarde (T)", "Noche (N)", "Descanso"], loc="upper center",
              bbox_to_anchor=(0.5, -0.02), ncol=4, frameon=False)
    ax.set_title(f"Mejor horario encontrado · {etiqueta}", fontsize=11, fontweight="bold", loc="left", pad=22)
    fig.tight_layout(); fig.savefig(os.path.join(FIG, "horario_optimo.png"), dpi=150); plt.close(fig)


def grafica_sensibilidad(sens):
    fig, ax = plt.subplots(figsize=(7.4, 3.8))
    x = [f"{s['indpb']}" for s in sens]
    b = ax.bar(x, [s["media"] for s in sens], yerr=[s["desv"] for s in sens], color=[VERDE2, VERDE, LIMA][:len(sens)],
               capsize=5, width=0.55)
    ax.bar_label(b, fmt="%.1f", padding=8, fontsize=9)
    ax.set_xlabel("Probabilidad de mutación por gen (indpb)"); ax.set_ylabel("Penalización final (media ± desv.)")
    ax.set_title("Sensibilidad a la tasa de mutación (10 corridas cada una)", fontsize=11, fontweight="bold", loc="left")
    ax.spines[["top", "right"]].set_visible(False)
    fig.tight_layout(); fig.savefig(os.path.join(FIG, "sensibilidad.png"), dpi=150); plt.close(fig)


def main():
    mejor, hist = evolucionar(42)
    corridas = [evolucionar(s)[1] for s in range(10)]
    finales = [c[-1]["mejor"] for c in corridas]
    evals = POBLACION + GENERACIONES * (POBLACION - ELITE)
    aleatorio = [busqueda_aleatoria(s, evals) for s in range(10)]
    pen = penalizaciones(mejor)

    print("=== Algoritmo genético (DEAP) · turnos de 8 empleados, 7 días ===")
    print(f"Población={POBLACION}  generaciones={GENERACIONES}  cxpb={CXPB}  mutpb={MUTPB}  indpb={INDPB}  torneo={TORNEO}  élite={ELIT
    print(f"Corrida semilla 42: penalización inicial (mejor) = {hist[0]['mejor']:.1f} -> final = {hist[-1]['mejor']:.1f}")
    print(f"10 corridas AG       : media = {np.mean(finales):.1f}  desv = {np.std(finales):.1f}  mejor = {min(finales):.1f}  peor = {ma
    print(f"10 búsquedas aleatorias ({evals} evaluaciones): media = {np.mean(aleatorio):.1f}  mejor = {min(aleatorio):.1f}")
    print("Desglose de la mejor solución (valor sin ponderar):")
    for k, v in pen.items():
        print(f"  {k:15} {v:6.2f}  (peso {PESOS[k]})")
```

```
230
231     sens = []
232     for indpb in (0.01, 0.04, 0.10):
233         f = [evolucionar(s, indpb=indpb)[1][-1]["mejor"] for s in range(10)]
234         sens.append({"indpb": indpb, "media": float(np.mean(f)), "desv": float(np.std(f))})
235         print(f"indpb={indpb}: media final = {np.mean(f):.1f} ± {np.std(f):.1f}")
236
237     grafica_convergencia(hist, corridas)
238     grafica_horario(mejor, f"penalización {hist[-1]['mejor']:.1f}")
239     grafica_sensibilidad(sens)
240
241     with open(os.path.join(AQUI, "resultados_s6.json"), "w", encoding="utf-8") as f:
242         json.dump({"hist": hist, "finales": finales, "aleatorio": aleatorio, "evaluaciones": evals,
243                    "desglose": pen, "sens": sens, "mejor": [int(g) for g in mejor],
244                    "inicial_mejor": hist[0]["mejor"]}, f, ensure_ascii=False, indent=1)
245
246
247 if __name__ == "__main__":
248     main()
```

SECCIÓN B

# Anexo B. Salida de la ejecución

```
=== Algoritmo genético (DEAP) · turnos de 8 empleados, 7 días ===
Población=100  generaciones=30  cxpb=0.8  mutpb=0.3  indpb=0.04  torneo=3  élite=2
Corrida semilla 42: penalización inicial (mejor) = 87.6 -> final = 8.5
10 corridas AG       : media = 3.1  desv = 3.7  mejor = 0.4  peor = 9.1
10 búsquedas aleatorias (3040 evaluaciones): media = 50.7  mejor = 41.4
Desglose de la mejor solución (valor sin ponderar):
  cobertura         0.00  (peso 10)
  sobrecobertura    0.00  (peso 2)
  dias_max          1.00  (peso 8)
  dias_min          0.00  (peso 3)
  noche_manana      0.00  (peso 6)
  sin_noche         0.00  (peso 15)
  noches_max        0.00  (peso 5)
  equidad_noches    0.50  (peso 1)
indpb=0.01: media final = 10.4 ± 6.5
indpb=0.04: media final = 3.1 ± 3.7
indpb=0.1: media final = 8.1 ± 4.2
```